OLMo 2/3 真开源

1. OLMo 2
   1.1. Normalization Layer Placement
   1.2. QK-Norm
2. Olmo 3
   2.1. Olmo 3 的模型流
      2.1.1. 基座模型训练
      2.1.2. 后训练
   2.2. Olmo 3 Base 训练细节
   2.3. Olmo 3 Think 训练细节
   2.4. Olmo 3 Instruct 训练细节
   2.5. Olmo 3 RL-Zero
   2.6. 怎么训练推理模型 - Nathan Lambert 经验分享

1. OLMo 2

> 论文链接：https://arxiv.org/abs/2501.00656

由非营利机构 Allen Institute for AI 发布的 **OLMo 系列模型**，因其在训练数据、代码和技术报告方面的高度透明而备受关注。

OLMo 模型结构清晰、设计规范，更重要的是，由于极高的透明度，它们为大语言模型的开发提供了极佳的参考范式。

在 OLMo 2 中有哪些值得注意的架构设计选择呢?

主要集中在归一化策略上：包括 RMSNorm 层的位置安排，以及 QK-norm（Query-Key 归一化） 的引入。

另一个值得一提的是，OLMo 2 **仍采用传统的多头注意力机制（MHA）**，并未采用 MLA 或 GQA 等新型注意力结构。

1.1. Normalization Layer Placement

总体而言，OLMo 2 在架构上大体沿用了最初 GPT 模型的设计，与当前多数主流大型语言模型相似。但它也存在一些值得关注的不同之处，先从归一化层的设计说起。

与 Llama、Gemma 以及大多数现代 LLM 一样，OLMo 2 **将归一化方法从 LayerNorm 替换为 RMSNorm**。

真正值得讨论的是 RMSNorm 的位置选择。在最初的 Transformer 架构中，两个归一化层分别位于注意力模块和前馈模块之后，这种结构被称为 Post-LN 或 后归一化（Post-Norm）。

而 GPT 及其后大多数的 LLM 模型，则将归一化层放在注意力模块和前馈模块的之前，这种做法称为 Pre-LN 或 前归一化（Pre-Norm）。

下图展示了 Post-Norm 与 Pre-Norm 的结构对比：

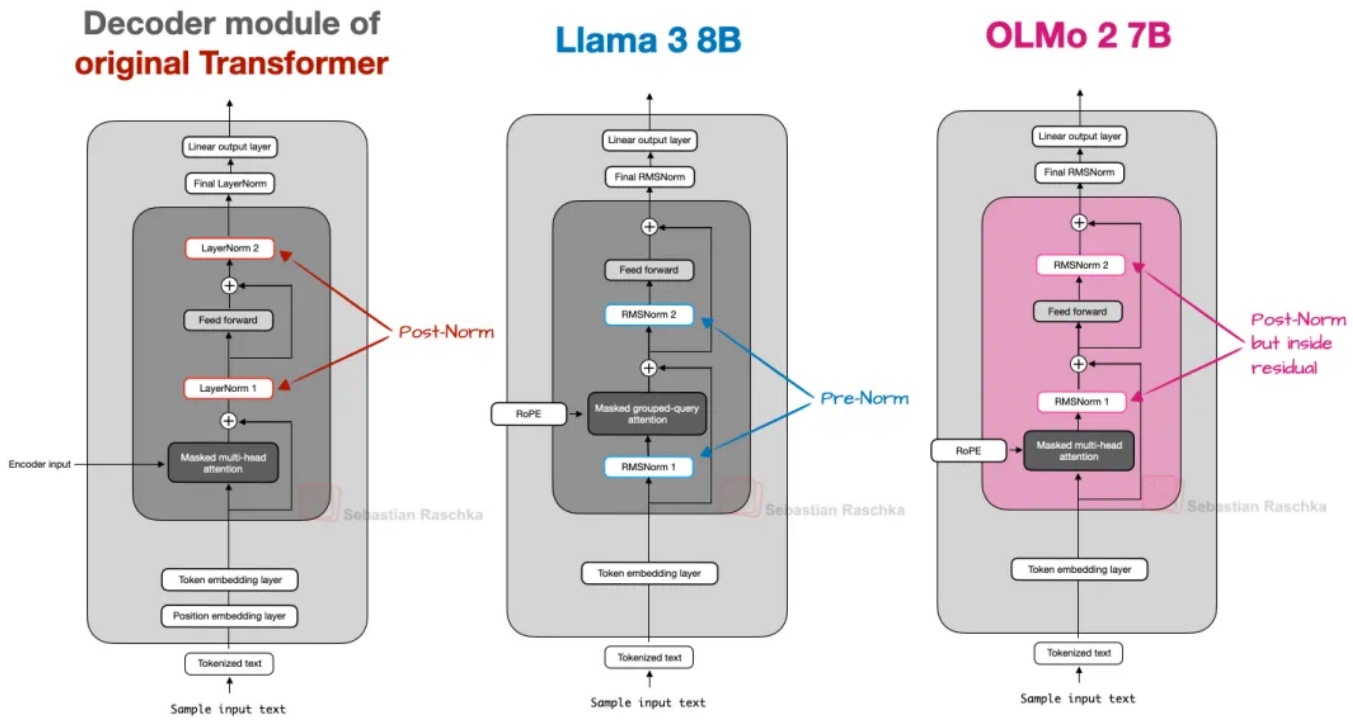


图 8：Post-Norm、Pre-Norm 以及 OLMo 2 采用的 Post-Norm 变体的对比图。

早在 2020 年，Xiong 等人就指出 Pre-LN 在模型初始化时可以带来更稳定的梯度。此外，研究人员还提到，Pre-LN 即使在不使用学习率预热的情况下也能正常训练，而这是 Post-LN 通常难以做到的。

在 OLMo 2 中，归一化层并不是放在注意力层和前馈网络之前，而是放在之后，如上图所示。然而，与最初 Transformer 架构不同的是，这些归一化层仍然嵌套在残差层内部。

那么，他们为何要调整归一化层的位置呢?

原因在于这种设计有助于提升训练的稳定性，这一点将在下图中进行展示。

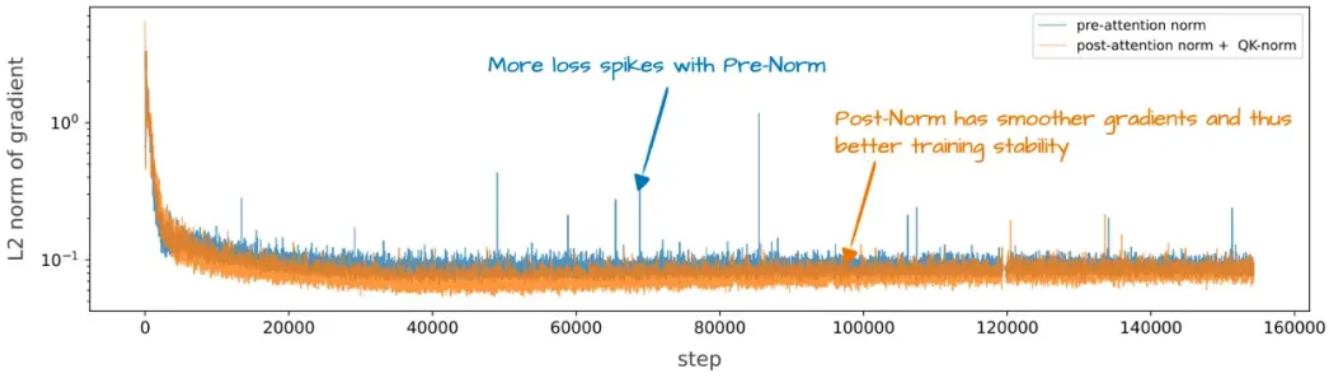


图 9：展示了 Pre-Norm（如 GPT-2、Llama 3 等所采用）与 OLMo 2 所采用的 Post-Norm 变体在训练稳定性方面的对比。

不过遗憾的是，这张图展示的结果同时包含了归一化顺序调整和 QK-Norm 两个因素，而后者是一个独立的概念。因此很难明确判断，归一化位置的改变对训练稳定性的提升到底贡献了多少。

1.2. QK-Norm

QK-Norm 本质上是另一个 RMSNorm 层，它被放置在 多头注意力模块内部，在应用旋转位置编码（RoPE）之前，对 Query 和 Key 进行归一化处理。

如前所述，**QK-Norm 与 Post-Norm 结合使用，有助于稳定训练过程**。关于 QK-Norm 更多细节，请参阅以下论文：https://arxiv.org/abs/2106.04560

简而言之，OLMo 2 架构中的主要设计亮点是 **RMSNorm 的放置方式**：将 RMSNorm 放置在注意力模块和前馈模块之后（属于 Post-Norm 的一种变体），并在注意力机制中对 query 和 key 引入额外的 RMSNorm（即 QK-Norm）。这两项改动结合使用，有助于稳定训练损失。

> 虽然 QK-Norm 缓解了训练过程中的数值不稳定性并被广泛采用，但它可能削弱模型的长上下文建模能力。后续 Olmo 3 会解释....在长上下文评估中，QK-Norm 表现最差，尽管它在其他能力上表现尚可

下图展示了 OLMo 2 与 Llama 3 的架构对比；可以看到，除了 OLMo 2 仍使用传统的 MHA 而非 GQA 外，二者在整体结构上相对接近。

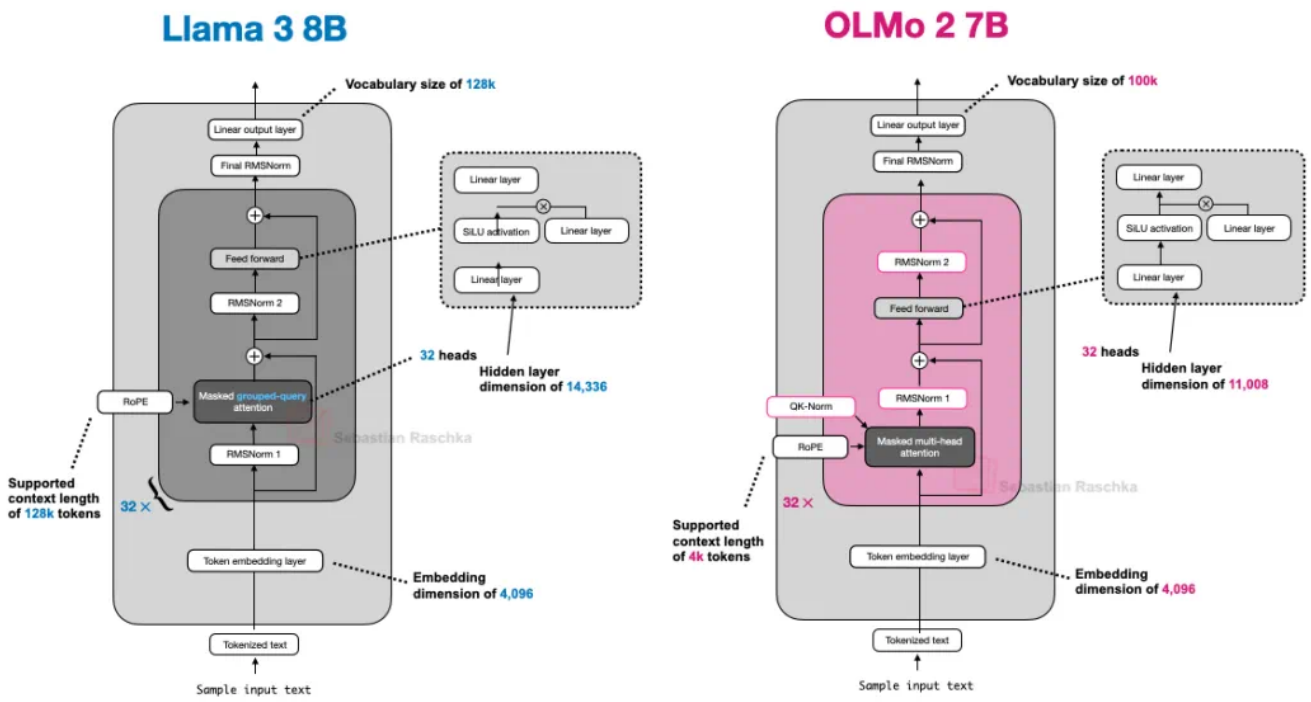


图 10：Llama 3 与 OLMo 2 的架构对比图。

2. Olmo 3

> **OLMo 生态系统项目** https://github.com/allenai/OLMo-core/
>
> 从零实现 Olmo 3 代码：Olmo 3 from-scratch implementation

Olmo 3

Olmo Team[★]

Allyson Ettinger[♥1] **Amanda Bertsch**[♥1,3] **Bailey Kuehl**[♥1] **David Graham**[♥1]
David Heineman[♥1] **Dirk Groeneveld**[♥1] **Faeze Brahman**[♥1] **Finbarr Timbers**[♥1]
Hamish Ivison[♥1,2] **Jacob Morrison**[♥1,2] **Jake Poznanski**[♥1] **Kyle Lo**[♥1,2] **Luca Soldaini**[♥1]
Matt Jordan[♥1] **Mayee Chen**[♥1,4] **Michael Noukhovitch**[♥1,5,6] **Nathan Lambert**[♥1]
Pete Walsh[♥1] **Pradeep Dasigi**[♥1] **Robert Berry**[♥1] **Saumya Malik**[♥1] **Saurabh Shah**[♥1]
Scott Geng[♥1,2] **Shane Arora**[♥1] **Shashank Gupta**[♥1] **Taira Anderson**[♥1] **Teng Xiao**[♥1]
Tyler Murray[♥1] **Tyler Romero**[♥1] **Victoria Graf**[♥1,2]

Akari Asai[1,3] **Akshita Bhagia**[1] **Alex Wettig**[7] **Alisa Liu**[2] **Aman Rangapur**[1]
Chloe Anastasiades[1] **Costa Huang**[1] **Dustin Schwenk**[1] **Harsh Trivedi**[1] **Ian Magnusson**[1,2]
Jaron Lochner[1] **Jiacheng Liu**[1] **Lj Miranda**[1] **Maarten Sap**[1,3] **Malia Morgan**[1]
Michael Schmitz[1] **Michal Guerquin**[1] **Michael Wilson**[1] **Regan Huff**[1] **Ronan Le Bras**[1]
Rui Xin[2] **Rulin Shao**[2] **Sam Skjonsberg**[1] **Shannon Zejiang Shen**[8] **Shuyue Stella Li**[2]
Tucker Wilde[1] **Valentina Pyatkin**[1] **Will Merrill**[1] **Yapei Chang**[9] **Yuling Gu**[1] **Zhiyuan Zeng**[1,2]

Ashish Sabharwal[1] **Luke Zettlemoyer**[2] **Pang Wei Koh**[1,2]
Ali Farhadi[1,2] **Noah A. Smith**[♥1,2] **Hannaneh Hajishirzi**[♥1,2]

[1]Allen Institute for AI [2]University of Washington [3]Carnegie Mellon University [4]Stanford University [5]Mila
[6]Université de Montréal [7]Princeton University [8]Massachusetts Institute of Technology [9]University of Maryland

[★]Olmo 3 was a team effort; authors sorted alphabetically. [♥]marks core contributors.

🤗 **Olmo 3 Base:** `Olmo-3-1025-7B` `Olmo-3-1125-32B`
🤗 **Olmo 3 Think:** `Olmo-3-7B-Think` `Olmo-3-32B-Think`
🤗 **Olmo 3 Instruct:** `Olmo-3-7B-Instruct`
🤗 **Olmo 3 RL Zero:** `Olmo-3-7B-RLZero-Math`, `Olmo-3-7B-RLZero-Code`, `Olmo-3-7B-RLZero-IF`, `Olmo-3-7B-RLZero-Mix`

Base Data: `Pretrain: Dolma 3 Mix` `Midtrain: Dolma 3 Dolmino Mix` `Long-ctx: Dolma 3 Longmino Mix`

Think Data: `SFT: Dolci-Think-SFT` `DPO: Dolci-Think-DPO` `RL: Dolci-Think-RL`

Instruct Data: `SFT: Dolci-Instruct-SFT` `DPO: Dolci-Instruct-DPO` `RL: Dolci-Instruct-RL-7B`

RL Zero Data: `Dolci-RLZero-Math-7B` `Dolci-RLZero-Code-7B` `Dolci-RLZero-IF-7B` `Dolci-RLZero-Mix-7B`

Training Code: `OLMo-core` (pretrain) `Open Instruct` (posttrain)
Data Code: `datamap-rs` (data processing) `duplodocus` (deduplication) `dolma3` (data recipes)
Eval Code: `OLMES` (eval suite)

Demo: 32B Think 7B Think 7B Instruct

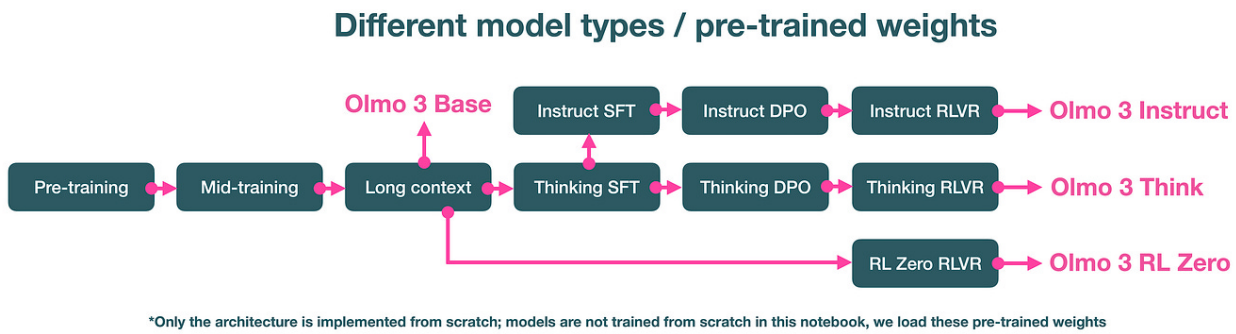
Contact: `olmo@allenai.org`

Allen AI 于 11 月 20 日发布了新的 Olmo 3 7B 和 32B 模型。（官方拼写已从 OLMo 更改为 Olmo，本节将采用这一拼写。）本次涵盖了 Base（基座）、Think（推理/思维链）、Instruct（指令遵循）以及 RL-Zero 版本。

- **OLMo 3 Think（旗舰模型）**：这是 OLMo 3 的旗舰推理模型，采用了 SFT –> DPO –> RLVR（带验证奖励的强化学习）的三阶段后训练配方。报告详细披露了如何通过“Delta Learning”构建偏好数据，以及 OlmoRL 框架在算法和基础设施上的改进（如去除了 KL 散度项、引入 Token 级损失等）。

与以往的开源模型不同，OLMo 3 的核心价值不仅仅在于发布了具有竞争力的模型权重，更在于其通过提供完整的**模型流 (Model Flow)**，重新定义了开源 AI 研究的透明度标准。

- **全阶段数据**：不仅公开了预训练数据，还公开了中期训练（Mid-training）、长上下文扩展以及后训练（Post-training）阶段使用的所有数据集。
- **全流程代码**：提供了从数据处理、预训练、微调到强化学习的完整代码库（OLMo-core, OlmoRL）。
- **中间产物**：发布了训练过程中的所有中间检查点（Checkpoints）、训练日志以及相关的依赖项。



First, let's take a look at the smaller of the two, Olmo 3 7B.

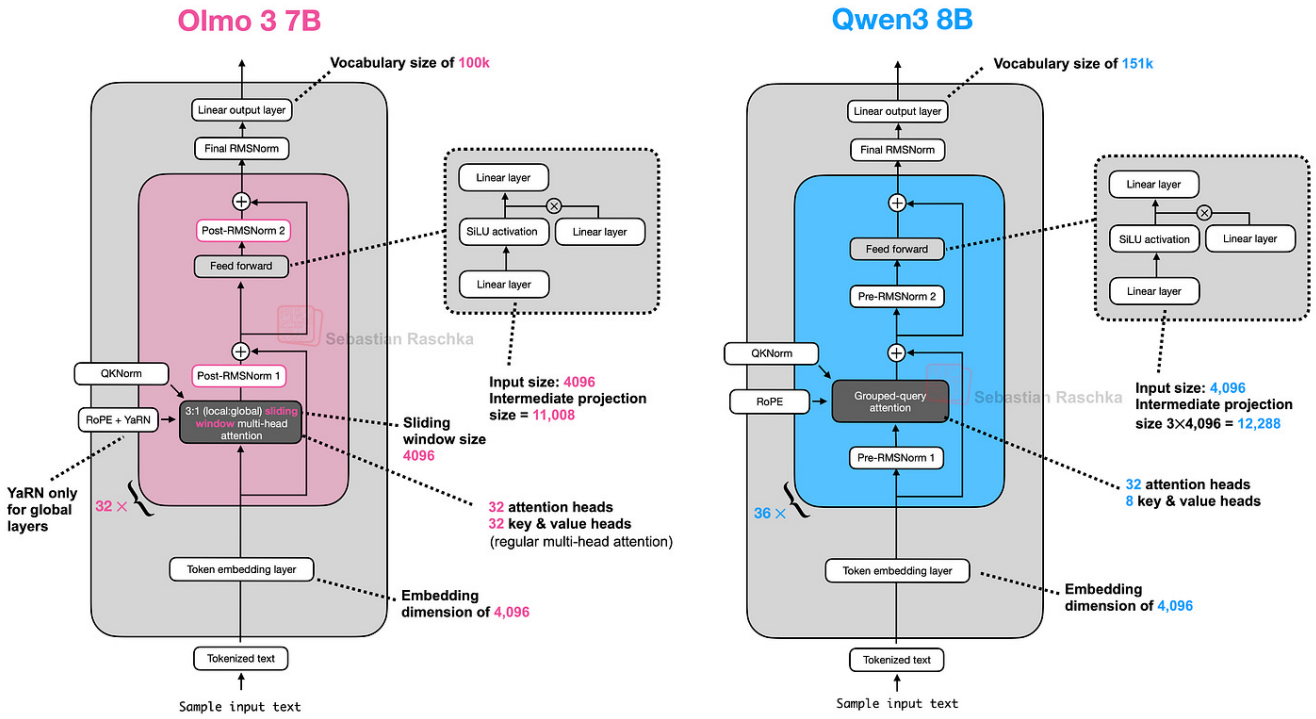


图 44：Olmo 3 7B 和 Qwen3 8B 并排对比。

如我们所见，Olmo 3 的架构与 Qwen3 相对相似。但值得注意的是，这很可能主要受 Olmo 2 前辈的启发，而不是 Qwen3。

与 Olmo 2 类似，Olmo 3 仍然使用后规范化而不是前规范化，因为在 Olmo 2 的论文中他们发现后规范化可以稳定训练。

有趣的是，7B 模型仍然使用类似于 Olmo 2 的多头注意力机制。然而，为了提高效率并缩小 KV 缓存大小，他们现在使用滑动窗口注意力（例如，类似于 Gemma 3）。

Next, let's look at the 32B model.

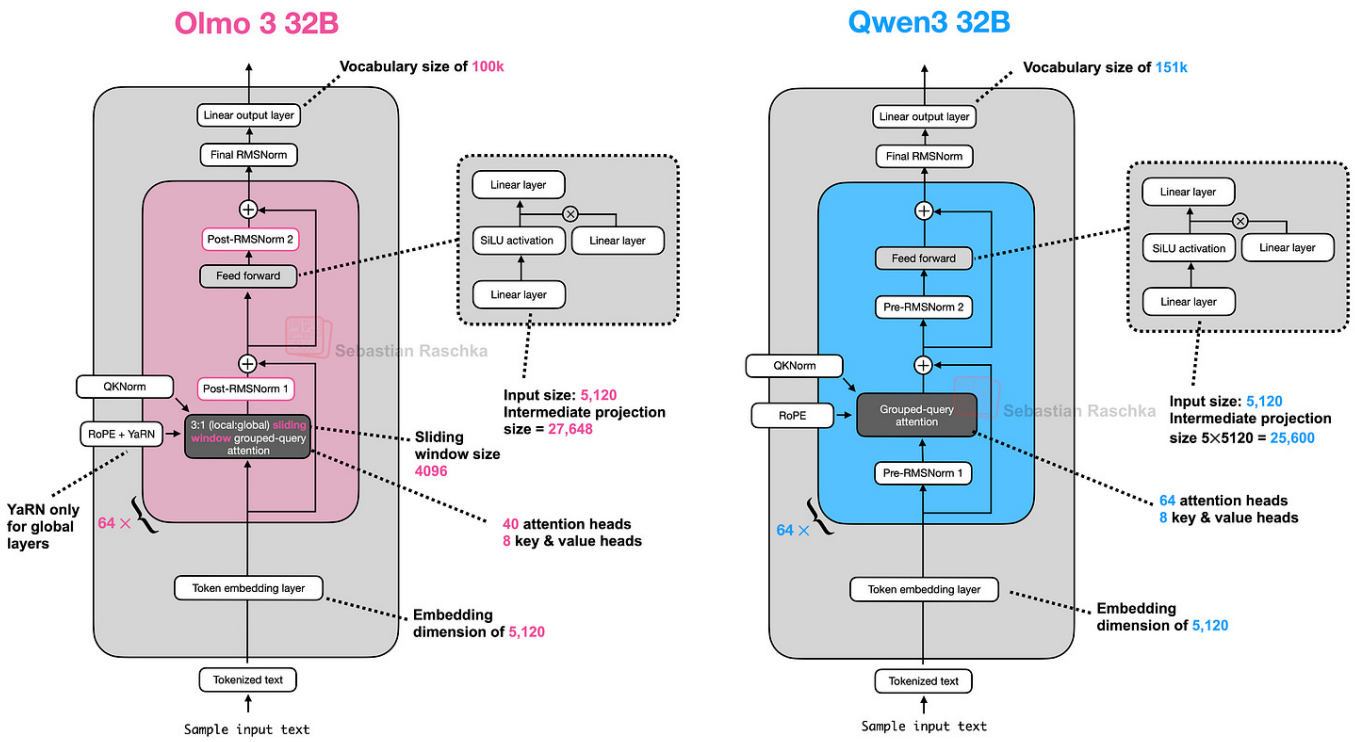


Figure 45: Olmo 3 32B and Qwen3 32B side by side.

图 45：Olmo 3 32B 和 Qwen3 32B 并排对比。

总体来说，这是相同的架构，只是规模更大。此外，比例（例如，从输入到前馈层中间大小的转换，等等）大致与 Qwen3 中的比例相匹配。

我的猜测是，由于词汇量较小，该架构最初比 Qwen3 要小一些，然后他们将 Qwen 3 中的中间大小扩展从 5 倍增加到 Olmo 3 中的 5.4 倍，以拥有一个 32B 模型进行直接比较。

另外，请注意 32B 模型使用了分组查询注意力–GQA。

也许一个小细节是，Olmo 3 使用 YaRN 进行上下文扩展，以支持 64k 的上下文长度，但仅用于全局（非滑动窗口注意力）层。（YaRN 本质上是一种仔细的 RoPE 重缩放技术，有助于在长上下文大小下更好地保持模型质量。）

在 Qwen3 中，YaRN 是可选的，用于将原生上下文从 32k 个 token 扩展到 131k 个 token。

2.1. Olmo 3 的模型流

本节详细概述了 Olmo 3 的完整模型流（Model Flow）。与仅发布最终模型权重的传统做法不同，OLMo 3 旨在通过提供模型全生命周期的访问权限——从初始概念到最终状态的每一个阶段、检查点、数据点和依赖项——来推动开源 AI 的研究。这种透明度允许研究者在模型开发过程的任意阶段（而不仅仅是最终权重）进行干预和定制。

模型流主要分为**基座模型训练**和**后训练**两个主要阶段，每个阶段包含特定的子阶段、数据配方和训练方法。

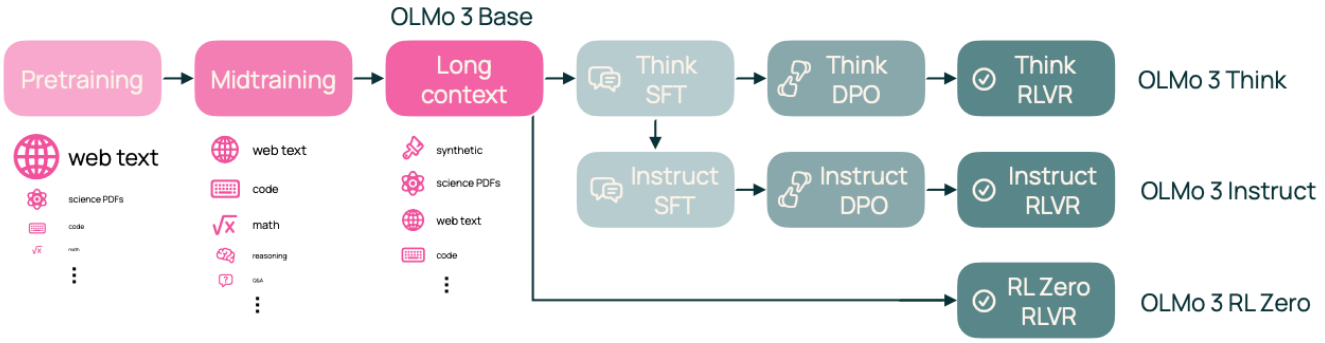


Figure 2 Depiction of OLMO 3's model flow. Development is divided into major pretraining and post-training stages, each further divided into sub-stages with their own recipes (i.e., training data and method).

OLMo 3 模型流示意图：展示了从预训练、中期训练、长上下文扩展到不同后训练路径（Think, Instruct, RL-Zero）的完整流程

2.1.1. 基座模型训练

OLMo 3 Base 的开发过程由三个连续的阶段组成：

1. **预训练 (Pretraining)**：规模最大，长达 5.9T tokens。
2. **中期训练 (Midtraining)**：规模为 100B tokens，用于特定能力的强化。
3. **长上下文扩展 (Long-context extension)**：针对 7B 模型为 50B tokens，32B 模型为 100B tokens，用于扩展上下文窗口。

评估体系

为了在基座模型开发（预训练和中期训练）期间做出高效的决策，团队开发了 **OlmoBaseEval** 基准套件。其核心目标是在计算资源受限的情况下（即基于小规模模型训练结果）对大规模训练效果进行预测。由于小规模模型在某些任务上可能表现出随机猜测的性能，导致基准测试噪声难以与模型改进区分，因此该套件采用了以下策略：

- **任务聚类**：将评估类似能力的基准测试（如多个代码任务或多个数学任务）聚类，通过聚合分数来减少方差。
- **代理指标**：针对小规模模型开发了特定的代理指标，侧重于识别训练过程中能力的“涌现”时

刻，而非仅看最终分数。

Bits-Per-Byte (BPB)。即计算模型生成“标准答案（Gold Label）”的负对数似然，并按答案的字节长度进行归一化。即使一个小模型无法生成正确的代码或数学答案（导致 Pass@1 为 0），它对正确答案赋予的概率可能会随着数据质量的提高而上升（即 BPB 下降）。这种连续指标能在能力“涌现”之前提供更平滑、更灵敏的性能信号。

- **信噪比优化**：通过在含噪任务上评估更多样本，或者直接剔除信噪比过低的任务，来提高评估信号的清晰度。

数据课程

每个训练阶段都配备了专门策划的数据集（统称为 **Dolma 3**），随着训练的进行，数据内容越来越侧重于核心能力，以便为下游任务（如推理、指令遵循）的激发做准备。

- **预训练 (Dolma 3 Mix)**：
  - 这是 OLMo 3 Base 的初始训练集，包含约 6T tokens。
  - 数据来源包括网页、学术 PDF、代码库等。
  - **技术创新**：
    i. 开发了新的工具链，支持万亿 token 规模的快速、可扩展全局去重。
    ii. 引入了两种优化训练 token 选择的新方法：**Token 约束混合 (token-constrained mixing)** 和 **质量感知上采样 (quality-aware upsampling)**。
    iii. 引入了新的学术 PDF 数据源——**olmOCR Science PDFs**，这些 PDF 使用 olmOCR 工具转换为线性化的纯文本，保留了高质量的科学内容。
- **中期训练 (Dolma 3 Dolmino Mix)**：
  - 这是一个 100B tokens 的数据集，专门用于在代码、数学和通用知识问答（QA）领域提升模型能力。
  - **方法论框架**：采用两部分流程。首先是**分布式反馈循环**，通过“微退火（microannealing）”快速评估单个数据源的有效性；其次是**中心化集成测试**，评估候选数据混合对基座模型质量和后训练可塑性的综合影响。
  - **数据构成**：有意识地包含了指令数据和思维链（thinking traces）数据，旨在为后续的 Think 和 Instruct 模型训练奠定基础。
- **长上下文扩展 (Dolma 3 Longmino Mix)**：
  - 该阶段使用了高质量的长文本数据。核心组件是 olmOCR 处理后的科学 PDF 集合。
  - 该集合包含超过 450 万个长度超过 32K 的文档，总计超过 3800 亿 tokens，是目前公开可用的最大长上下文研究数据集之一。

- 通过此阶段，OLMo 3 将上下文窗口扩展至 64K。

公开产物 (Open Artifacts)

Ai2 发布了每个阶段结束时的模型以及所有中间检查点。在数据方面，不仅发布了**数据混合 (Data Mixes)**（即模型实际训练所用的 tokens），还发布了**数据池 (Data Pools)**（经过清洗的 10T tokens 源数据）。为了方便社区在有限计算资源下进行实验，还提供了缩小版的数据混合样本（例如 100B tokens 的 Dolma 3 Mix 样本）。

2.1.2. 后训练

如图 2 所示，OLMo 3 的后训练流程包含三条主要路径，分别对应 **OLMo 3 Think**、**OLMo 3 Instruct** 和 **OLMo 3 RL-Zero**。此阶段引入了新的后训练数据套件 **Dolci** 和算法改进 **OlmoRL**。

- **OLMo 3 Think**：
  - **目标**：通过生成扩展的思维链（Thinking Traces）来执行分步推理，最终得出答案。
  - **训练流程**：采用三阶段训练：SFT →**偏好微调 (DPO)** →带验证奖励的强化学习 (RLVR)。
  - **数据 (Dolci Think)**：包含用于 SFT 的长思维链合成示例；用于 DPO 的高质量对比数据（基于 Delta Learning 构建，相对差异（delta）比“选中”答案本身的绝对质量更为关键）；以及用于 RL 的高难度提示词。
  - **算法 (OlmoRL)**：针对长思维链的 RLVR 进行了算法和基础设施层面的优化，将验证性推理从单一领域扩展到了数学、代码和通用对话等多个领域，实现了 4 倍的训练加速。
- **OLMo 3 Instruct**：
  - **目标**：生成高效、有用且简洁的回复，专注于通用对话和工具使用（Function Calling），不生成内部思维链。
  - **设计考量**：优先考虑用户需求，避免过度冗长，注重推理时的效率。
  - **数据 (Dolci Instruct)**：
    - **SFT**：富含专门为函数调用创建的数据。
    - **DPO**：引入多轮对话偏好数据，并实施针对性的**数据长度干预**，以鼓励模型生成简洁的响应。
    - **RL**：使用 RLVR 进一步微调核心能力，同时利用偏好调整来保持回复的简洁性。
- **OLMo 3 RL-Zero**：
  - **背景**：目前的开源 RLVR 基准通常基于未公开预训练数据的开放权重模型（如 Qwen, Llama 等），这导致社区难以研究预训练数据对 RL 性能的影响，且容易出现基准测试

泄漏（导致虚假奖励有效）或提示词工程干扰等问题。

- **方案**：发布 **Dolci RL-Zero** 数据集、算法配置和 **OlmoRL** 代码。
- **流程**：直接从 OLMo 3 Base 模型开始进行 RLVR 训练，涵盖数学、代码、精确指令遵循（IF）及混合领域。
- **严谨性**：对 Dolci RL-Zero 进行了针对预训练和中期训练数据的严格去污染（Decontamination）处理，确保 RLVR 效果的研究不受数据泄漏的混淆。

2.2. Olmo 3 Base 训练细节

OLMo 3 技术报告的解读，本文略长（2万字）

2.3. Olmo 3 Think 训练细节

OLMo 3 技术报告的解读，本文略长（2万字）

2.4. ~~Olmo 3 Instruct 训练细节~~

2.5. ~~Olmo 3 RL-Zero~~

2.6. 怎么训练推理模型 – Nathan Lambert 经验分享

如何训练一个开源推理模型Olmo-Thinking – 青稞

- **核心目的是为了让研究者能清晰地分辨：模型的性能提升，究竟是源于算法的**真实贡献**，还是仅仅因为它学会了利用数据中的**虚假关联**（spurious correlation）来‘应付’评测。
- **如何拿到最优的数据配比？**答案是，**你的数据选择，本质上反映了你对模型能力的期望。** 想要模型在推理上更强，一个直接的策略就是增加数学、科技和代码数据的比重。
- **长上下文的瓶颈？** Olmo 作者发现 QK Norm 的使用虽然可以让训练更加稳定，但限制了模型在长上下文中的推理能力。
- **后训练（Post-training）怎么做的？**
  - 通过推理蒸馏完成监督微调（SFT）：使用reasoning数据“蒸馏”的方式对小模型进行SFT，这对于提升小模型推理效果是十分高效的！
  - **Reasoning DPO**, it works! 作者解释，对于一些reward稀疏的任务是很难做RLVR的，

一般期望SFT去缓和，但SFT会对泛化有一定影响，而 thinking-dpo实际上提供了一个优秀的新思路。通过“**Delta Learning**”构建偏好数据，让答案之间的相对差异（delta）比“accept”答案本身的绝对质量更为关键。

    - 例，“选中”答案：由一个强大的模型（如 Qwen-3 32B）生成。“拒绝”答案：由一个能力稍弱的同系列模型生成（如 Qwen-2.5 0.6B）。
    - Hugging Face 发布的 smollm3 模型就是一个很好的参考，他们也采用了类似的DPO流程来增强模型能力。
  - RLVR 的工程挑战？RL 的大部分时间是浪费在推理上的，所以需要解耦生成与训练，通常的解法是**Asynchronous和Off-Policy**。但代价是牺牲了部分数值稳定性和理论收敛性，在**“Off-Policy”下，**生成器（vLLM）和训练器（如HF Trainer）中的模型状态不完全同步，导致它们各自的概率分布存在偏差。这些细微的数值差异会在训练中累积，构成主要的稳定性挑战。
    - 为了解决异步RL中的数值挑战，校准（生成器与训练器）之间的概率分布差异。
    - **1、 重新计算对数概率 (Re-calculating Log Probs)**：在vLLM生成token后，将其传递给训练器模型，重新计算一次对数概率。这种方法能确保梯度计算的数值精度，但代价是增加了额外的GPU计算开销，本质上是**用计算时间换取准确性**(我们现在的方法 很贵)
    - 2、**重要性采样 (Importance Sampling)**：通过重要性采样，可以将一个概率分布下的采样数据有效地用于另一个分布的期望计算。
- 作者最后提到，接下来的工作就是 RL Scaling，将这一套验证可行的方案扩展到更大的模型训练。